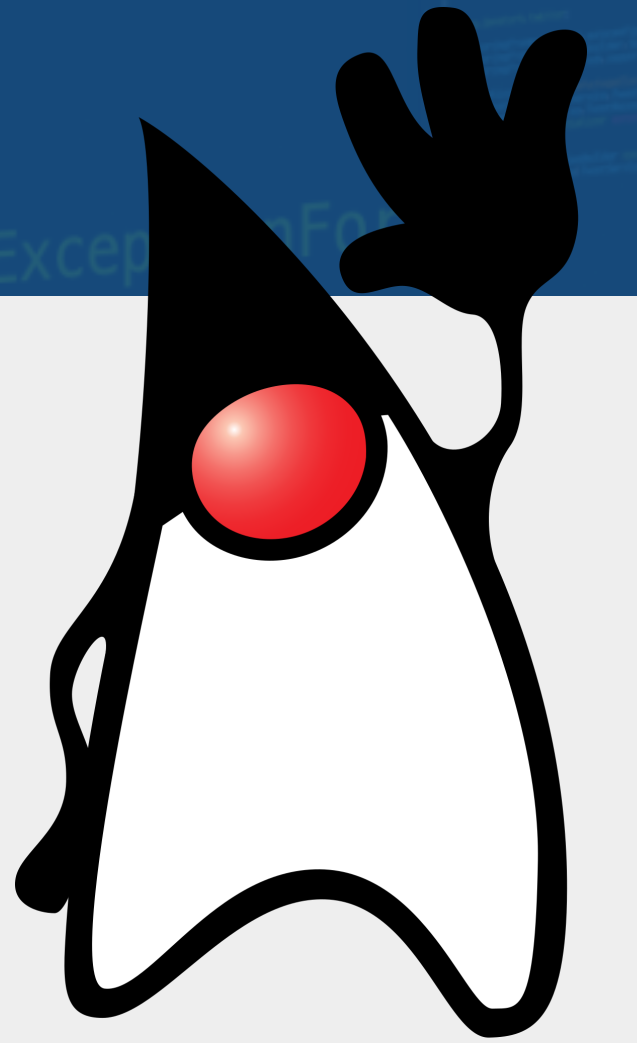


Java Programlama Dili

23. Bölüm

Sarmalama ve Bilgi Saklama

Akın Kaldıroğlu



Amaç



- Bu bölümde sarmalama (encapsulation) ve bilgi saklama (information hiding) ele alınacaktır
- Bu amaçla önce bu iki kavram ele alınacak, sonra da bu kavramları gerçekleştirecek mekanizmalar olan erişim yönetimi (access management) incelenecektir



- Sarmalama (Encapsulation)
- Bilgi Saklama (Information Hiding)
- Erişim Kontrolü
 - Sınıf Erişim Kontrolü
 - Üye Erişim Kontrolü
- get/set Metotları
- Ödevler
- Bundan Sonra?

Sarmalama (Encapsulation)



Sarmalama (Encapsulation) - I

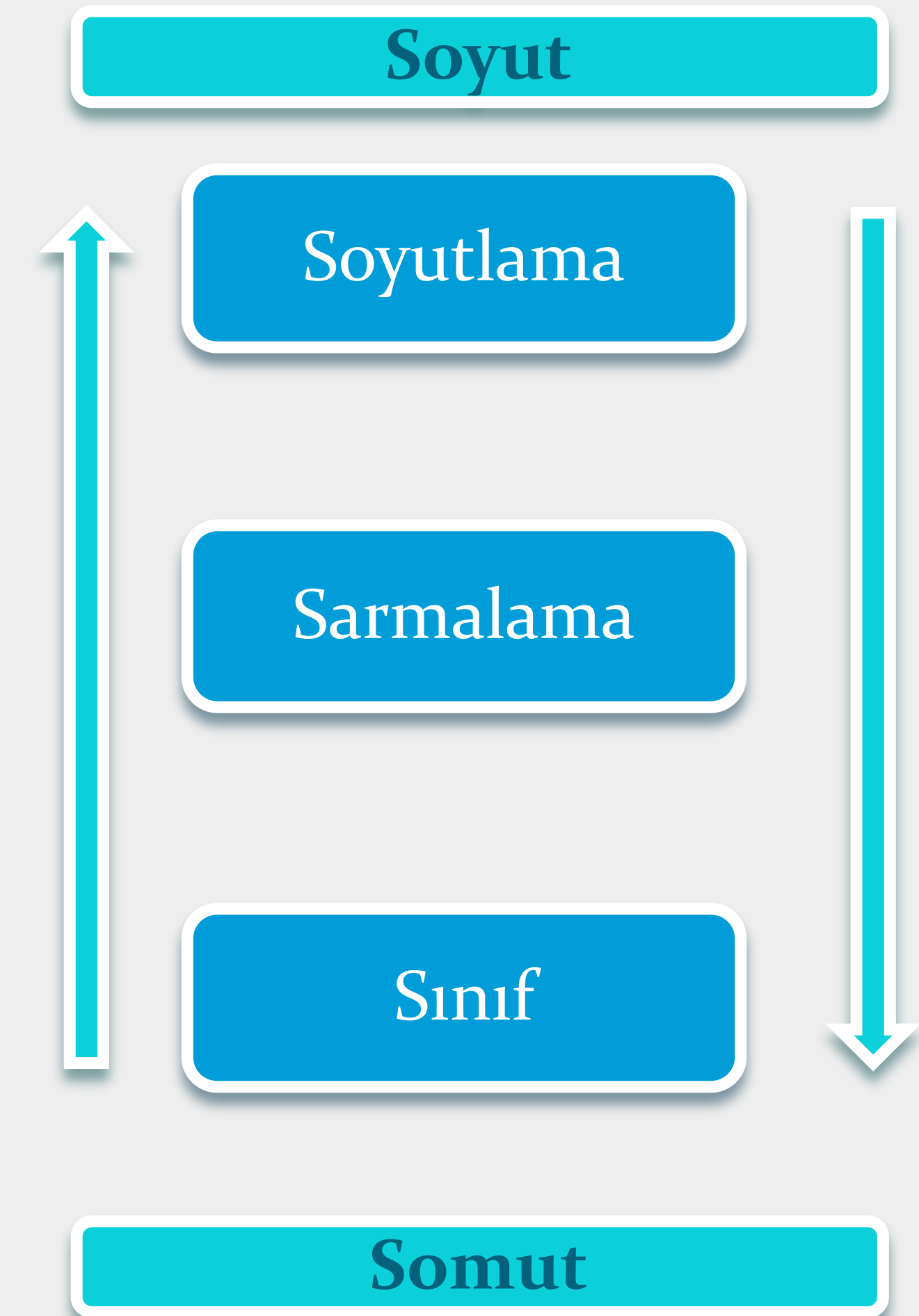


- Verinin, o veri üzerinde çalışacak davranışlarla birlikte bir yapı altında toplanmasına **encapsulation (sarmalama/bohçalama)** denir
- Sarmalama ya da encapsulation soyutlamanın tasarlanmış hali olarak düşünülebilir
- Java vb. dillerdeki sınıf (class), arayüz (interface), record vb. yapılar da, sarmalamanın gerçekleştirmeleridir
- Arayüz (interface), record vb. yapılar bu eğitimin konularından değildir

Sarmalama (Encapsulation) - II



- Nesne merkezli ya da nesne yönelimli (object-oriented) dillerde en temel sarmalama mekanizması sınıftır (class)
- Bazı dillerdeki *struct* da sarmalama yapısıdır
- Sınıflar, soyutlamaların gerçekleştirildiği yapılardır
- Sarmalama ile nesnelerin durumu (verileri) ile davranışları, sınıf ile mantıksal bir birim haline getirilir



Soyut ve Kullanıcı Tanımlı Veri Tipi



- Tarihi olarak programlama dillerinde sarmalama, **soyut veri tipi (abstract data type)** ya da **kullanıcı tanımlı veri tipi (user-defined data type)** adları altında var olan bir kavramdır
- Fakat bu yapılar nesne merkezli dillerin *nesnesinden* farklıdır
- Sınıflar (ve nesneleri) aynı zamanda birer soyut ve kullanıcı tanımlı veri tipidir ama bunlardan daha fazlasını sağlarlar

Bilgi Saklama (Information Hiding) - I

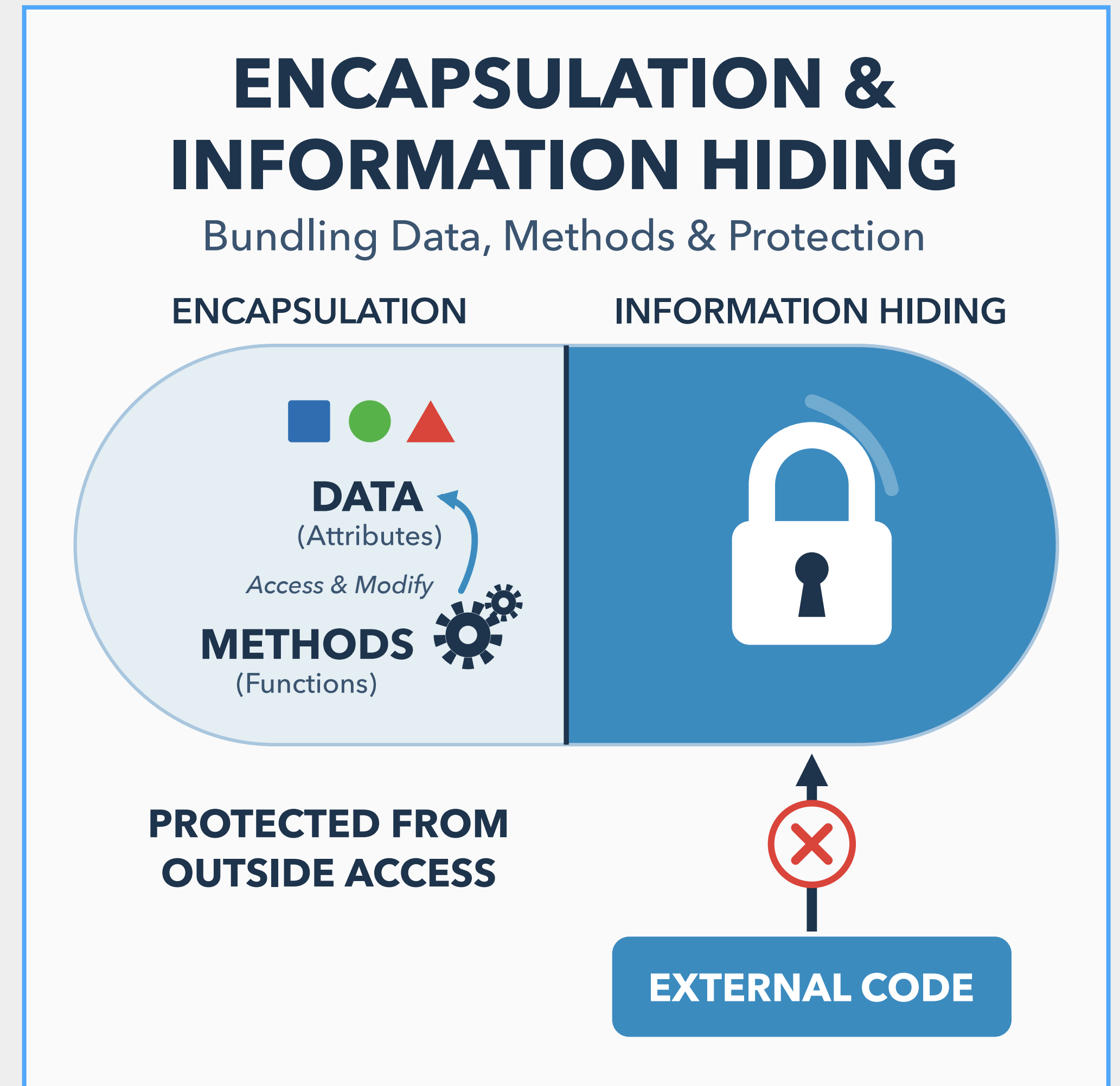


- Bu yapıya sarmalama (encapsulation) denmesinin sebebi, sarmalanan yapının iç mekanizmasının dışarıdan saklanması, kapsül içine alınarak yani kapsüllenenerek (encapsulation), dış erişimden korunmasıdır
- Bu durum ayrıca **bilgi saklama (information hiding)** ile de ifade edilir
- **Gerçekleştirme saklama (implementation hiding)** da denir

Bilgi Saklama (Information Hiding) - II



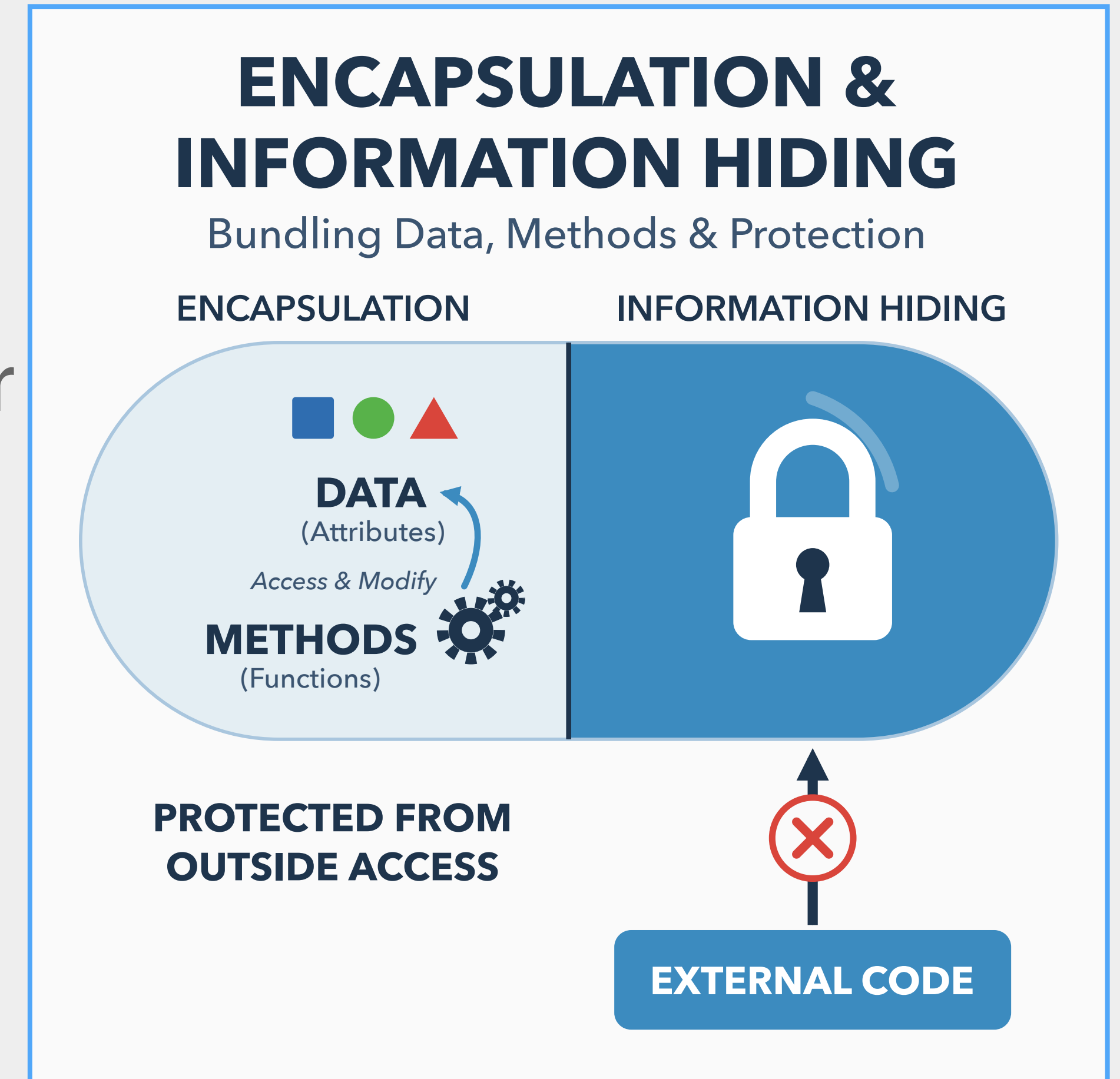
- Sarmalama, bilgi saklama prensibiyle birlikte, bir sınıfın iç yapısını dışarıdan saklarken, onun arayüzü üzerinden dışarıya yani istemcilerine (clients) hizmet vermesidir
- Dışarısı ya da dış dünyadan kasıt, istemcilerdir (clients), nesneden hizmet alan tüm diğer yapılardır



Bilgi Saklama (Information Hiding) - III



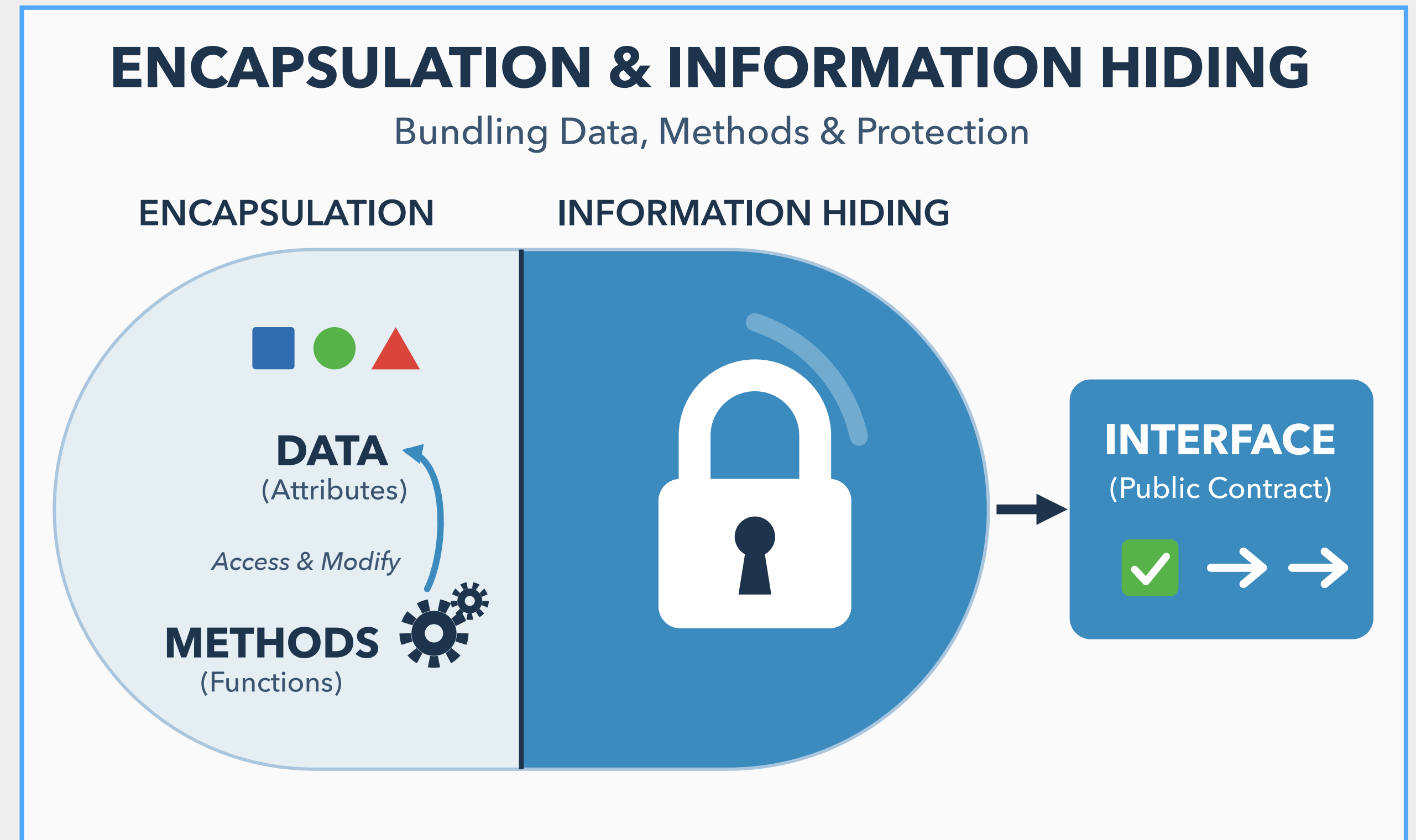
- Yani sınıflar dışarıya, istemcilerine hizmet sağlarken, sadece hizmet bilgisinin bilinmesini, o hizmeti oluşturan iç mekanizmaların saklı kalmasını tercih eder
- Çünkü aynı hizmet çok farklı iç mekanizmalarla verilebilir dolayısıyla iç mekanizmalar sık değişme eğilimindedir ve sarmalama ile bu değişimler istemcilerden habersiz olarak yapılabilir



Hizmet ve Gerçekleştirilmesi - I



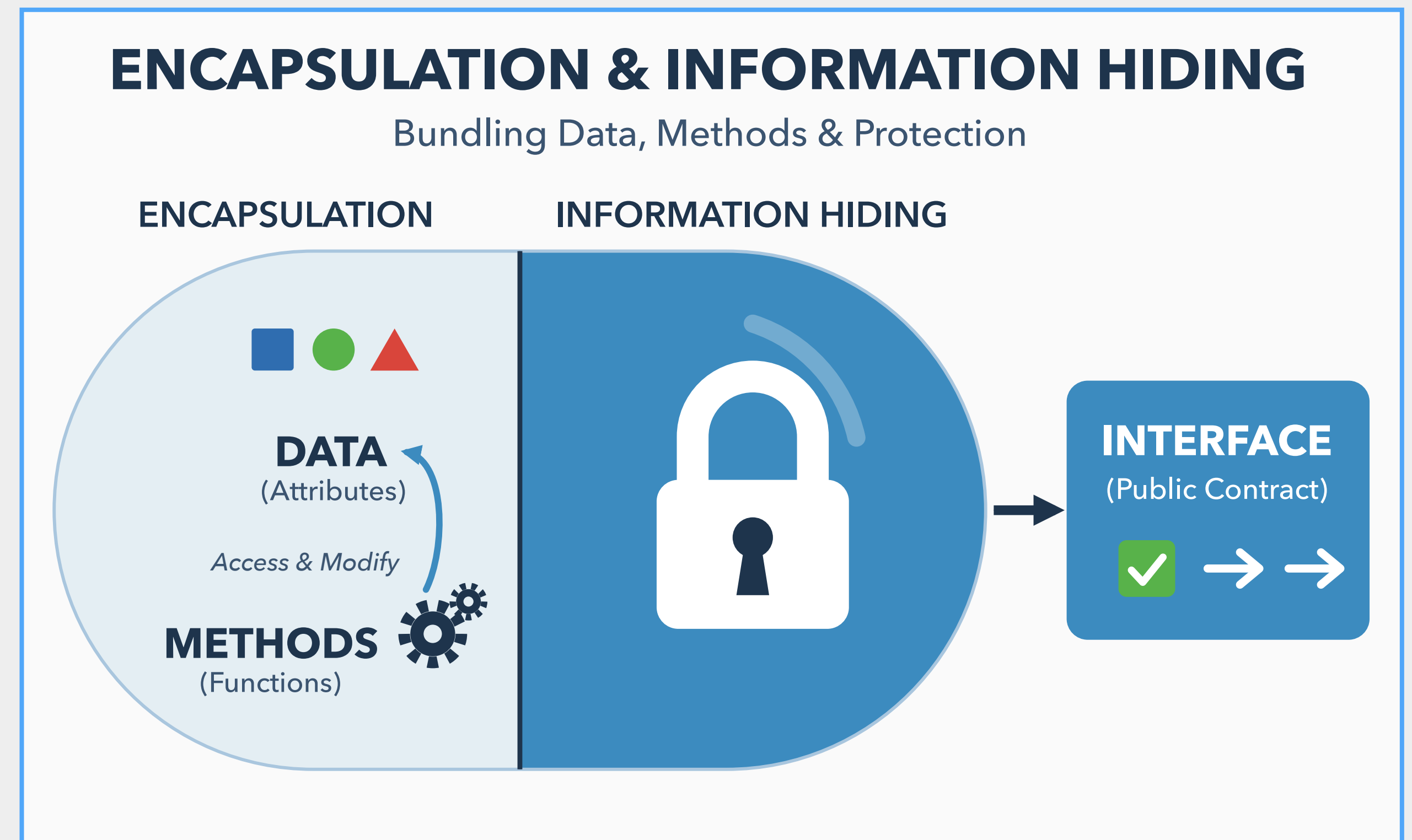
- Hizmet bilgisi **nesnenin arayüzü** (**object's interface**), hizmeti oluşturan mekanizma ise nesnenin gerçekleştirmesidir (**object's implementation**)
- İstemcilerin, bir nesneden hizmet almak için nesnenin hizmet bilgisini bilmesi yeterli olmalıdır



Hizmet ve Gerçekleştirilmesi - II



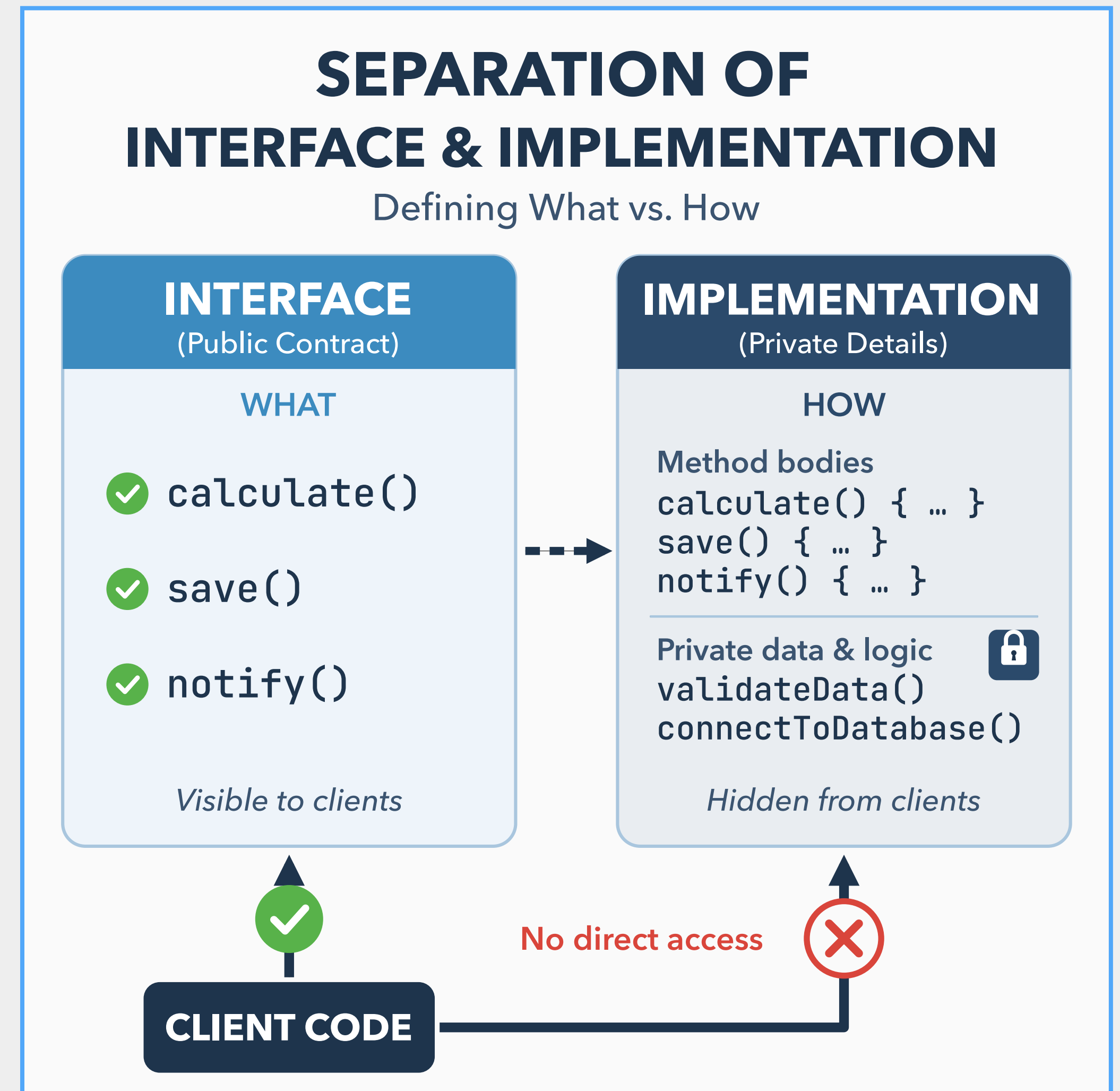
- İstemciler, bir nesneden hizmet almak için o hizmeti yerine getiren iç mekanizmalar hakkında bilgiye gerek duymamalıdır



Arayüz - Gerçekleştirme Ayrımı



- Buna **arayüz - gerçekleştirme ayrımı** (**separation of interface and implementation**) denir
- Arayüz, **nelik (what-ness)**, gerçekleştirme ise **nasıllık (how-ness)** olarak görülebilir
- Arayüz dıştan, istemciye görünen yüz, gerçekleştirme ise görünenin arkasındaki detaydır



Bağlılık - Coupling - I



- Dolayısıyla, nesneler arasındaki **bağlılık (coupling)** ya da **bağımlılık (dependency)**, sadece arayüz seviyesinde olmalı, gerçekleştirme seviyesinde olmamalıdır
- Bu prensip literatürde, veciz olarak farklı ifadelerle belirtilmiştir.

Bağlılık - Coupling - II



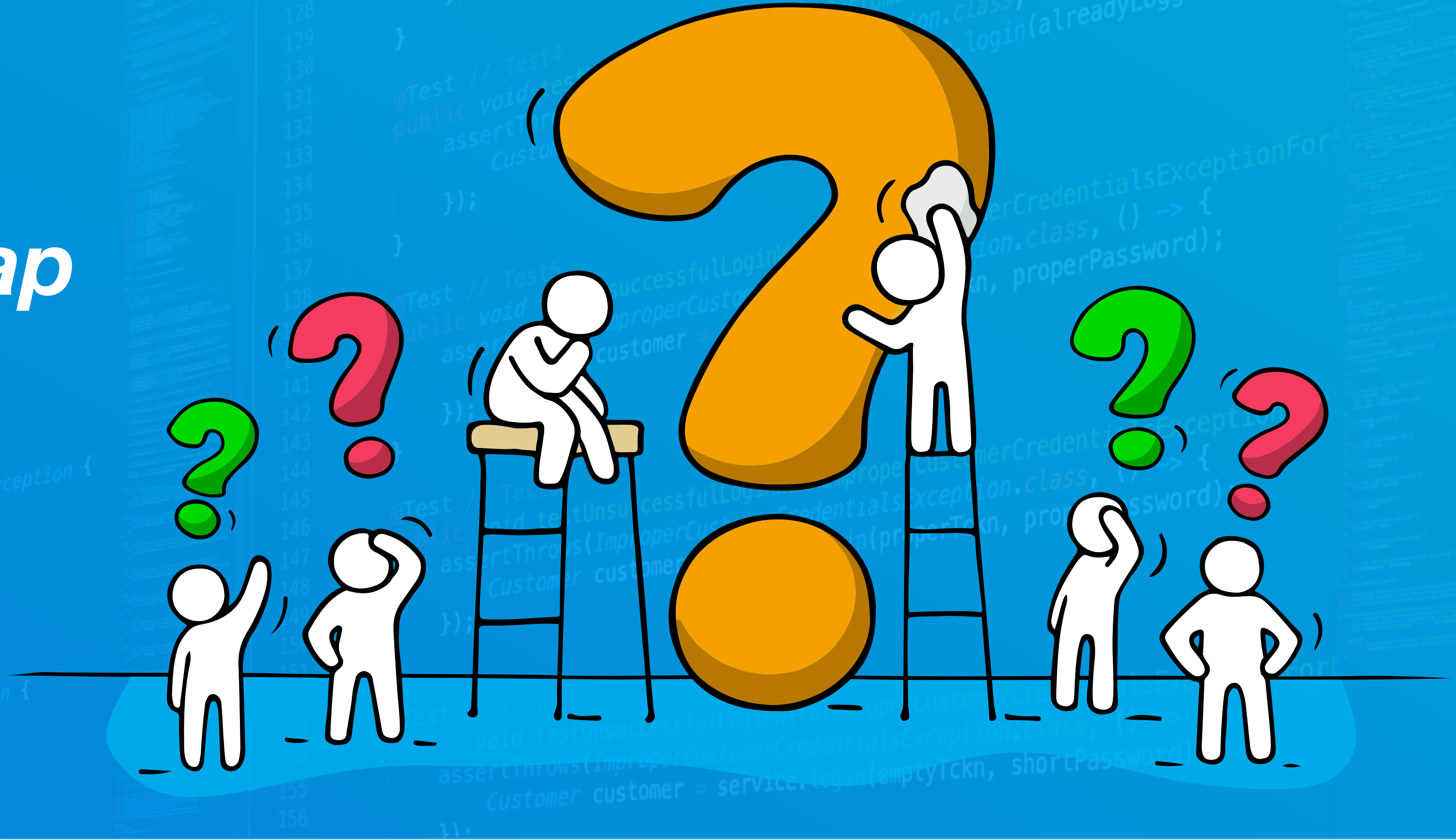
Program to an interface, not an implementation

- Erich Gamma ve arkadaşları (GoF), *Design Patterns* kitabında bunu ifade etmişlerdir

Depend on abstractions, do not depend on concretions

- Bu da Robert C. Martin (Uncle Bob) tarafından ifade edilmiştir
- Tüm bunlar 1970'lerde David L. Parnas'ın bilgi saklama üzerine yaptığı çalışmalara kadar geri götürülebilir

Soru ve Cevap Zamanı!



Bilgi Saklama (Information Hiding)



Sarmalama ve Bilgi Saklama



- Az önce bahsettik, sarmalamada iki yön söz konusudur:
 - birim olarak bir araya getirme ve
 - bu birimin iç yapılarını dışarıdan koruma
- İkinci yön, **bilgi saklama (information hiding)** ile ifade edilir ve **gerçekleştirme saklama (implementation hiding)** da denir
- Bu anlamda bilgi saklama, sarmalamanın tamamlayıcı özelliğidir

Bilgi/Gerçekleştirme Saklama



- Dolayısıyla bilgi/gerçekleştirme saklama prensibi, nesnelerin arayüzlerinin istemcilerine açık tutulurken, iç detaylarının yani gerçekleştirmelerinin dışarıdan korunması, saklanması gerektiğini ifade eder
- Bu durum aslen sadece nesneler için değil tüm sistemler için geçerlidir
- Ya da sadece nesneler için değil, irili ufaklı pek çok yazılım soyutlaması için geçerlidir

Neden Bilgi Saklama?



- Çünkü soyutlamalar, gerçekleştirmelerden daha az değişir

Abstractions are more stable than concretions

Arayüz - Interface



- Zaten **arayüz** ya da **interface**, anlam olarak arada olana, ara birime işaret eder
- Yani arayüz iki yapının birbirlerine dokundukları noktadır, birbirleriyle iletişimde bulunmalarını sağlayan birimdir
- Bu sebeple aradaki iletişimin sadece arayüz üzerinden olması ve gerçekleştirme detaylarına erişilmemesi ve bu detayların arayüze taşınmaması önemlidir

Birliktelik ve Bağlılık - I



- Sistemlerin iç yapısı tamamen o sistemin amacına yönelik, odaklı olmalı, bu amacı gerçekleştirirken de dışarıdan olabildiğince az yardım almalıdır
- Bir sistemin bir amaca odaklılığına **birliktelik (cohesion)**, dışı ile olan bağlantısına ise **bağlılık (coupling)** denir
- Sarmalama birlikteliğe vurgu yaparken bilgi saklama bağlılık ile ilgilidir

Birliktelik ve Bağlılık - II



- Sistemler ve alt parçalarının birliktelikleri yüksek, bağlılıkları düşük olmalıdır
- **Highly/strongly-cohesive and loosely-coupled**
- **High/strong-cohesion and loose/low-coupling**
- Yani sistem ve her bir alt parçası, kendini çok iyi bilirken, dışarıyı olabildiğince az bilmelidir

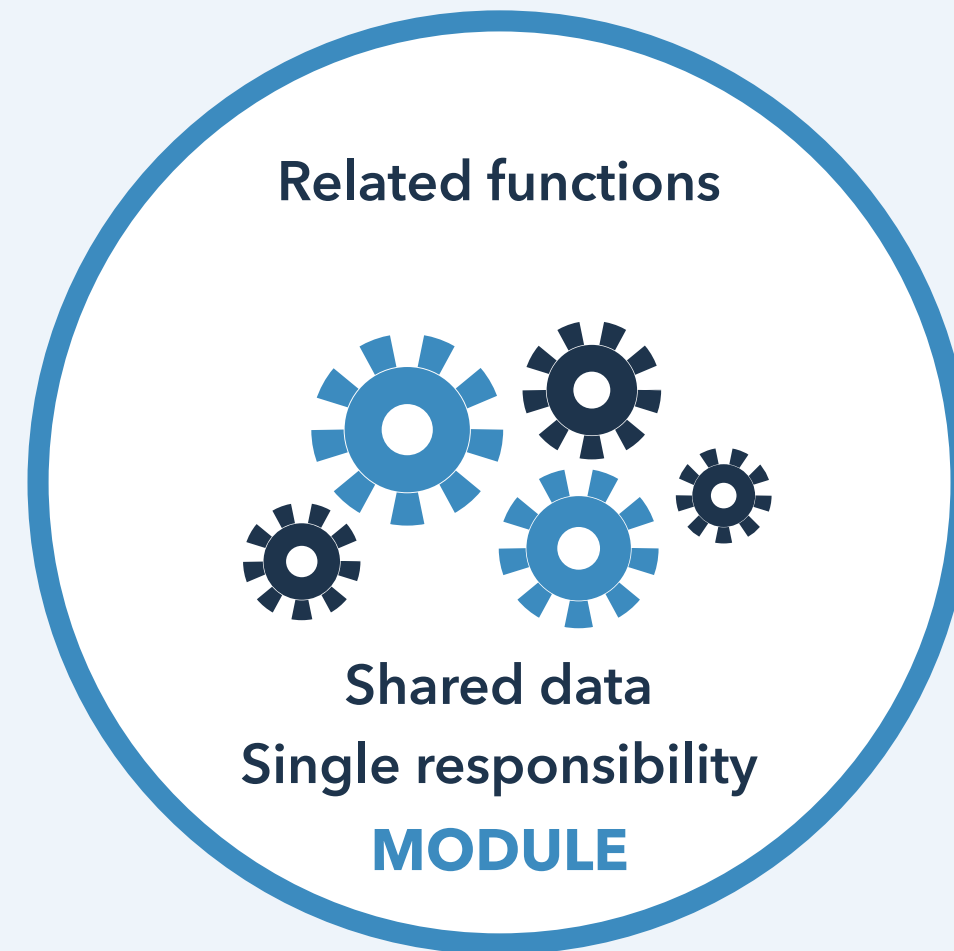
Birliktelik ve Bağlılık - III



COHESION & COUPLING

HIGH COHESION

Keep it high



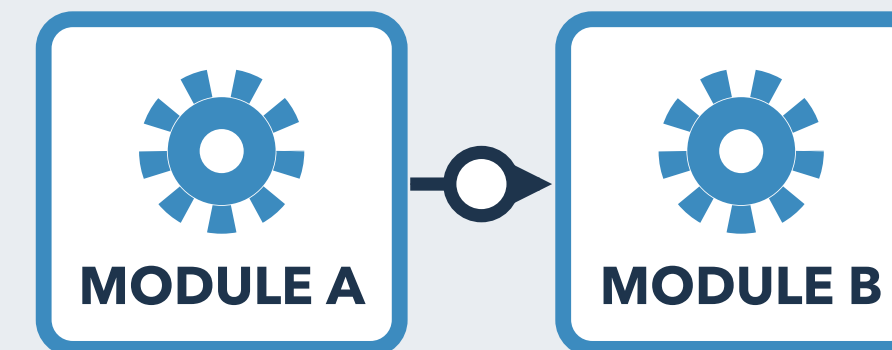
Everything in the module serves one purpose

Strong focus = easier to maintain

LOW COUPLING

Keep it low

One narrow interface



- ✓ Talk through interfaces
- ✗ No shared internal data
- ✗ No hidden dependencies

Modules stay independent
Loose connections = easier to change

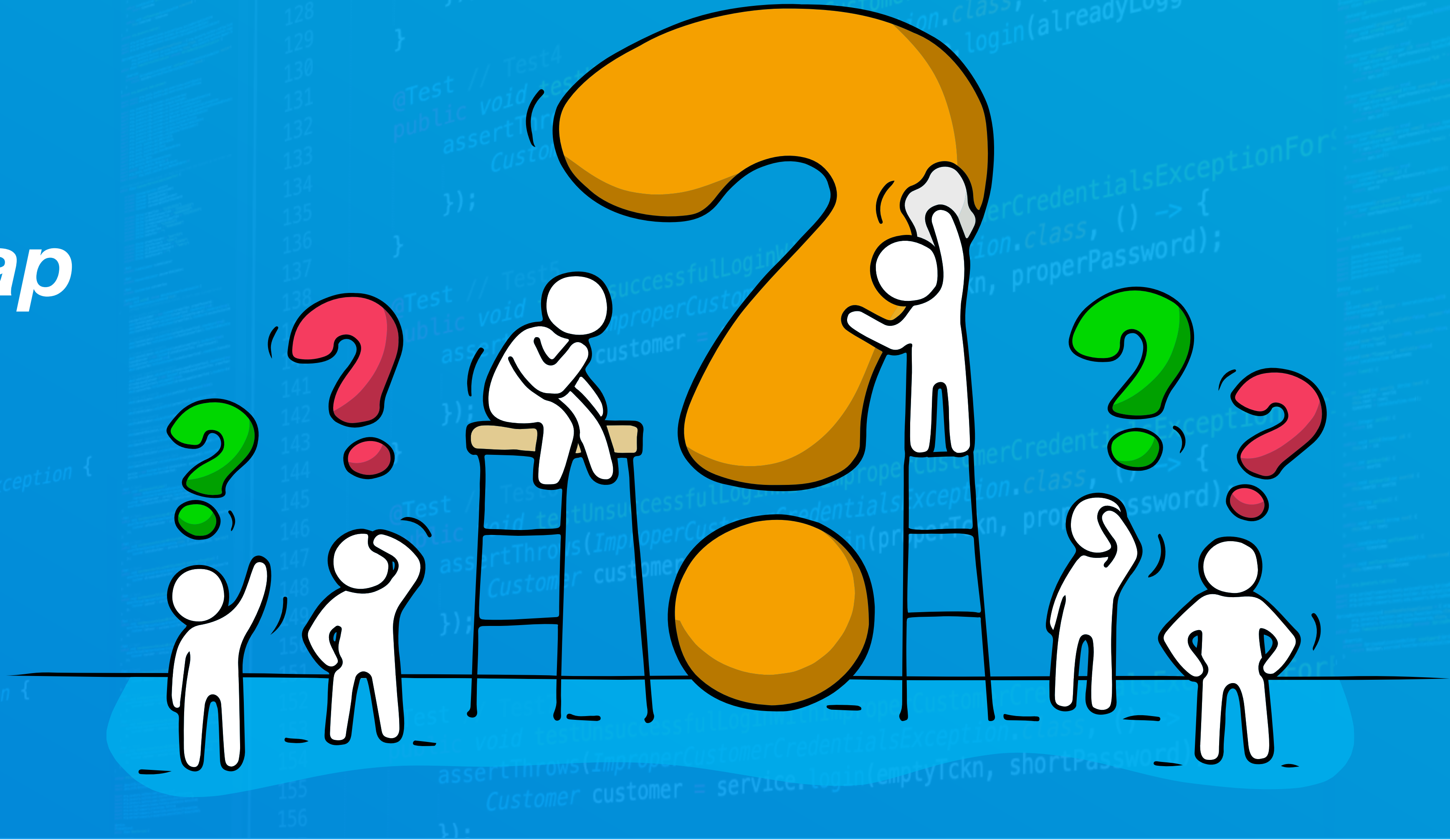
Goal: Maximize Cohesion, Minimize Coupling

API, Application Programming Interface



- **API, Application Programming Interface**'in kısaltmasıdır
- API, bir sistemin (ya da sınıf gibi bir alt parçasının) dışarıdan ulaşılabilen özelliklerine denir
- API, arkasında sakladığı detayın üzerine bina edilen, bir sınıfın ya da sistemin arayüzüdür
- İstemciler, bir sistem ya da sınıftan ancak API'si ile hizmet alabilir

Soru ve Cevap Zamanı!



Erişim Kontrolü



Erişim Kontrolü



- Bağılılığı düşük ve iç birlikteliği yüksek (loosely-coupled and highly-cohesive) sınıflar oluşturmak için nesne merkezli dillerde bilgi (ve gerçekleştirme) saklamayı sağlamaya **erişim kontrolü (access control)** denir
- Java'da erişim kontrolü **erişim niteleyiciler (access modifiers)** ile gerçekleştirilir

Erişim Niteleyiciler - I



- Orijinal halinde Java'da erişim kontrolü sadece paketlerdeki sınıflar ve sınıflardaki üyeler üzerinden yapılabilir
- Erişim niteleyiciler, sınıfların ve iç yapılarının dışarıdan erişimini kontrol etmek için kullanılır
- Java'da paketlere her zaman erişim vardır, onlara erişimi kontrol eden bir yapı yoktur
- Ama Java 9 ile birlikte gelen modül yapısıyla birlikte artık paketlere de erişim kontrol edilebilir



- Java'ya 9. sürüm ile birlikte, paketten daha yukarıda bir soyutlama yapısı olarak modüller (module) gelmiştir
- Modül, birbirleriyle ilgili paketlerden oluşan bir bileşik yapıdır
- Modüllerin bu bileşik yapısını ifade eden tanıtım dosyaları (descriptor file) vardır
- Modül, içerdiği paketlerin dışarıdan erişimini de kontrol etmeye izin verir
- Bu eğitimde modüller ele alınmayacak ve etkileri göz ardı edilecektir

Erişim Niteleyiciler - II



- Java'da iki tür erişim niteleyici vardır:
 - **Sınıf erişim niteleyicileri:** Sınıflara erişimi kontrol etmek içindir
 - **Üye erişim niteleyicileri:** Sınıfın ve nesnenin üyelerine yani alanlarıyla metotlarına erişimi kontrol etmek içindir; dışarıya açmak ya da dışarıdan saklamak üzere vardır
- Nesne ve sınıf üyeleri yanında kurucular için geçerlidir

Erişim Niteleyiciler - II



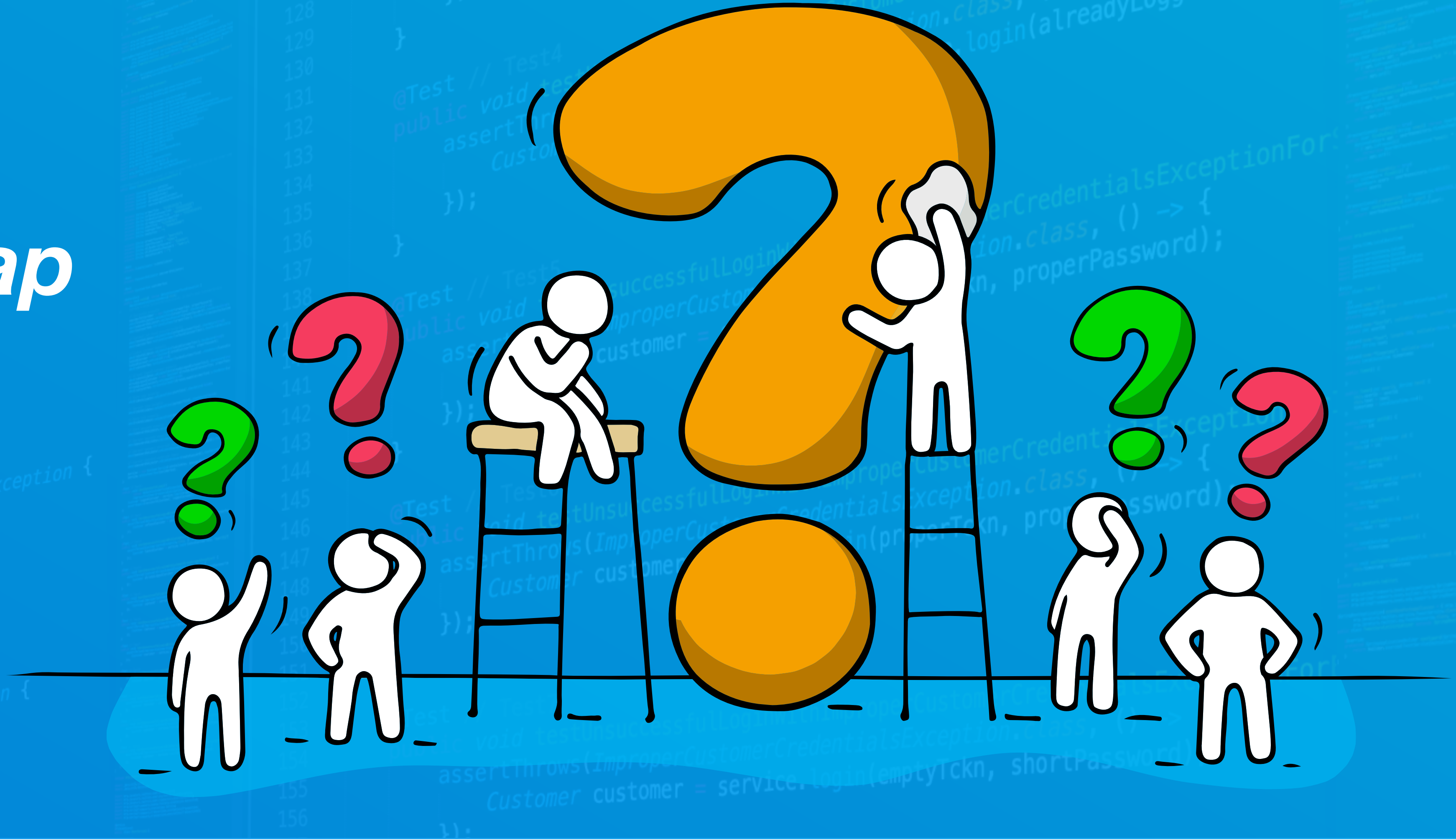
- Erişim niteleyiciler için 3 anahtar kelime vardır:
 - `public`
 - `private`
 - `protected`

Erişim Niteleyiciler - IV



- Dışarıya açılan sınıflar, sistemin API'sinde görünürler, açılmayan sınıflar görünmezler
- Dışarıya açılan üyeler, sınıfın API'sinde görünürler, açılmayan üyeler ise görünmezler

Soru ve Cevap Zamanı!





Sınıf Erişim Kontrolü

Sınıf Erişim Niteleyicileri



- Sınıflar için iki seviyeli erişim söz konusudur
- Bu iki seviye, **public** anahtar kelimesinin var olduğu ve olmadığı iki durumla belirlenir:
- **public**: Sınıfa her yerden erişilebilir
 - **public** sınıf kendi ismine sahip **java** kaynak dosyasında bulunmalıdır
- Varsayılan (**public** kelimesi yok): Bu durumda sınıfa sadece içinde bulunduğu paketten ulaşılır, dışından yani diğer paketlerden ulaşılamaz

Varsayılan Erişim



- Eğer bir sınıf ya da ileride ele alacağımız başka bir tip, **public** değilse **varsayılan erişime (default accessibility)** sahiptir
- Bu erişime **paket erişimi (package accessibility)** ya da **arkadaşça erişim (friendly access)** denir
- Paket erişimine sahip sınıflar, paketin dışından görülmezler, onlara sadece paketteki sınıflar ulaşabilir
- Ama **public** sınıflara, paket içinden ve dışından, yani her yerden erişilebilir

ClassA ve ClassB



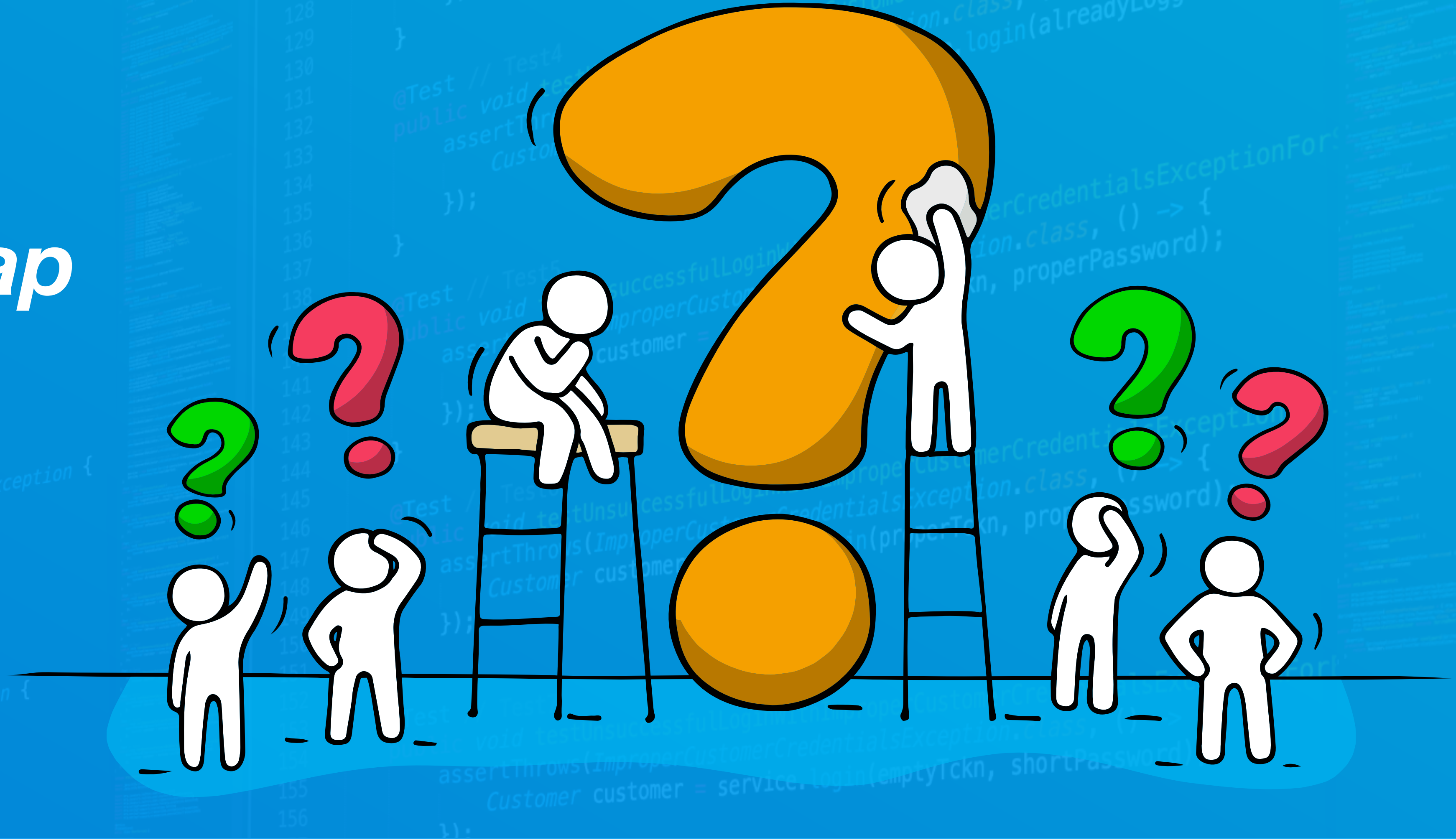
- `classAccess.ClassA` ve `ClassB`
- `ClassZZ`'ye aynı pakette olan `ClassXX` ulaşırken farklı pakette olan `ClassA` ulaşamaz

public ya da Varsayılan Erişim?



- **public** sınıflar, paketlerinden dışarıya açık olduklarından, sistemin API'sinde görünür, varsayılan erişimli yani *non-public* sınıflar ise görünmez
- Yani **public** sınıflar, API üzerinden dış dünya ile etkileşimde bulunur ve kendi paketi dışındaki istemcilere (client) hizmet sağlarken, paket erişimli sınıflar API'nin arka tarafındaki mekanizmaları oluşturur

Soru ve Cevap Zamanı!





Üye Erişim Kontrolü

Üye Erişim Kontrolü



- Üye erişim kontrolünden kasıt, bir sınıfın üye değişkenleri ile üye metotlarına erişimin kontrolüdür
- Yani sınıf değişken ve metotları ile nesne değişken ve metotlarına erişimin kontrolüdür
- Üye olmamasına rağmen kurucular da bu kontrole dahildir

Üye Erişim Niteleyicileri - I



- Bir sınıf ve nesne kendi üyelerine erişir, önemli olan dışarıdan yani başka sınıflardan ve nesnelerden erişimi kontrol etmektir
- Üyelere erişim için dört seviye vardır:
 - `public`
 - `protected`
 - Varsayılan (default), hiçbir niteleyicinin kullanılmadığı durumdur
 - `private`

Üye Erişim Niteleyicileri - II



- **public** olan üyelere her yerden erişilir
- **private** olan üyeler, dışarıdan erişime tamamen kapalıdır; sadece içinde bulunduğu sınıftan erişilir
- Varsayılan halde, erişim sadece paketteki sınıflara açıktır
- **protected** üyeler, miras olarak devralan alt sınıflara erişime açıktır
 - Aynı paketteki sınıflar da **protected** üyelere erişebilir

Üye Erişim Niteleyicileri - III



Niteleyici	Kendisi	Paketi	Çocukları (Farklı Pakette)	Herkes (Farklı Pakette)
public	+	+	+	+
protected	+	+	+	-
- (varsayılan)	+	+	-	-
private	+	-	-	-

ClassA



- `memberAccess.ClassA`

public Üyeler - I



- **public**: Genel olarak üye metotlar (member methods) **public** yapılır
 - Çünkü metotlar, nesnenin arayüzünü oluşturur
- Üye değişkenler (member variables) **public** yapılmaz
 - Çünkü üye değişkenler, nesnenin bilgisini ya da durumunu temsil eder ve bu bilginin saklanması amaçlanır (information hiding), bu sebeple genelde **private** olur

public Üyeler - II



- `static` ve `final` olan değişkenler, yani sabiteler, diğer sınıfların ihtiyaç duyduğu durumda varsayılan ya da `public` erişime sahip olabilir
- Paketteki diğer sınıfların kullanımına açmak için varsayılan, tüm sistemin kullanımına açmak için `public` yapılır
- Aksi takdirde `private` olarak kalır

protected Üyeler



- **protected**: Sadece miras devralan sınıfların (inheriting/child classes) miras olarak ulaşabilecekleri üyeler **protected** yapılırlar
- **protected**, aynı paketdeki sınıflara **public**, farklı paketdeki devralmayan sınıflara **private**'tir
- Java'da paketten koruyup, sadece devralan sınıflara açan bir erişim niteleyici yoktur

Varsayılan Üyeler



- **Varsayılan:** Varsayılan erişime (default access) sahip olan üyelere sadece paket içinden erişim söz konusudur, paket dışından onlara ulaşılamaz
- Varsayılan erişime, *pakete özel (package-private)* erişim de denir

private Üyeler - I



- **private**: Bu üyeler dışarıya tamamen kapalıdır
- **private** üyeler sınıfın mahrem, saklı, kendi kullanımına ait yapısıdır
- Üye değişkenler genel olarak **private** yapılır
 - Bilgi saklama prensibi!
- Sadece iç çalışma için oluşturulmuş, sınıftaki diğer metotlara hizmet veren dolayısıyla arayüzde olmaması gereken, *helper* metotlar da **private** yapılır

private Üyeler - II

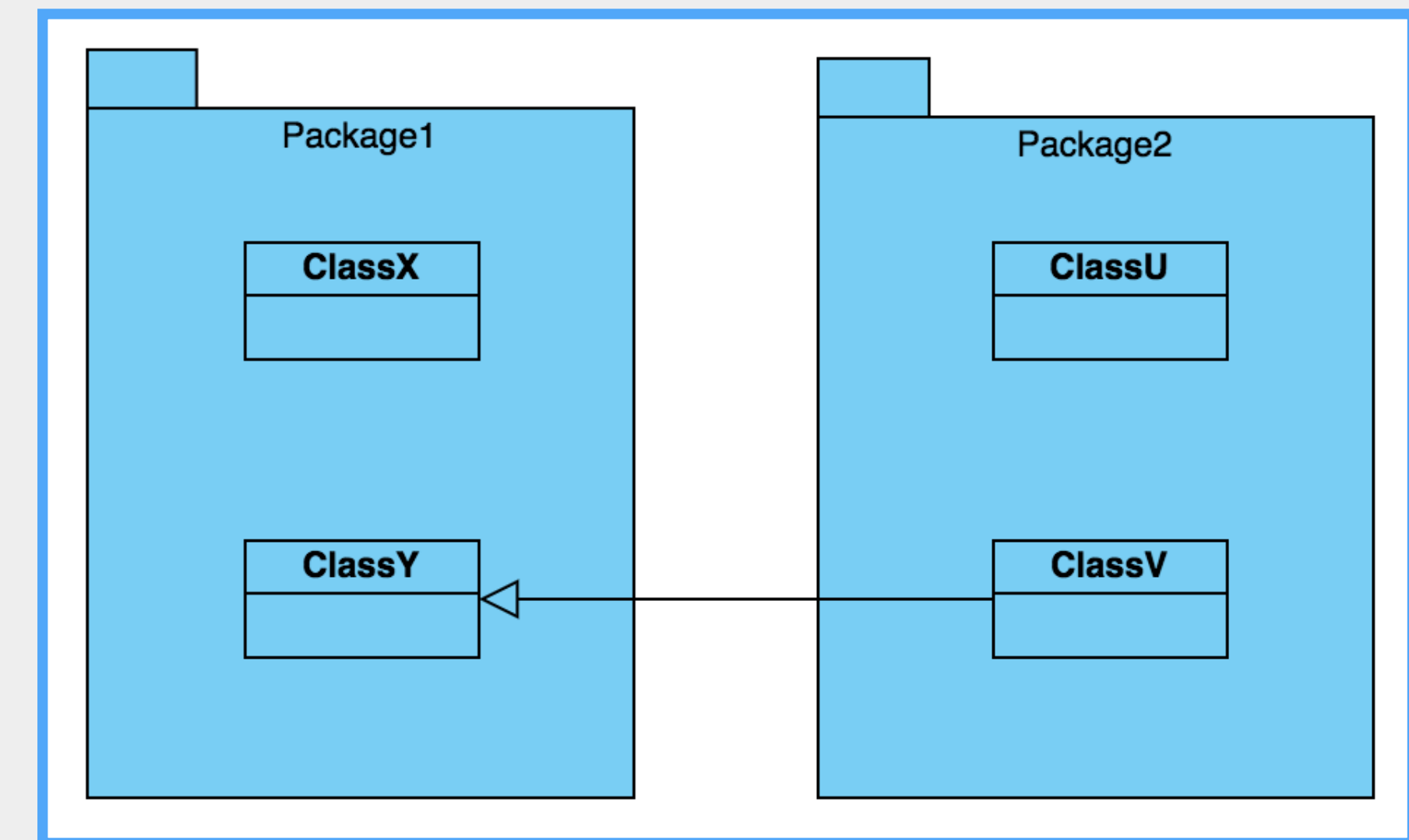


- Bir sınıftaki bir üyenin **private** olması, bu sınıfın nesnelerinin bu üyesine, farklı sınıflardan ve nesnelerinden ulaşamayacakları anlamına gelir
- Ama aynı sınıfın nesneleri, birbirlerinin **private** alanlarına ya da sınıfın **private** alanlarına ulaşabilir

Örnek - I



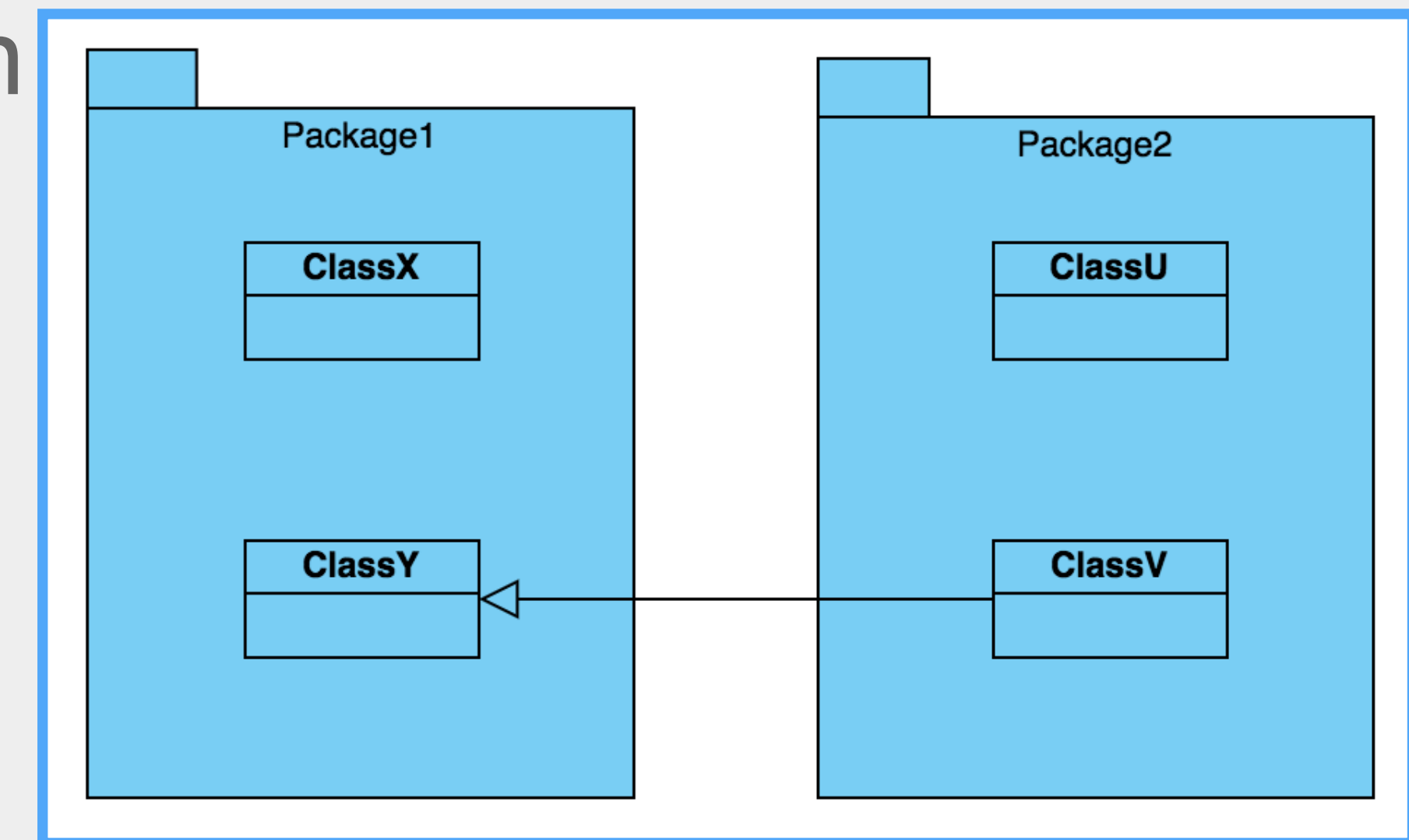
- Şu iki paket ve sınıflar için şunlar söylenebilir:
- **ClassU** ve **ClassV** sınıfları, **Package1**'deki sınıfların sadece **public static** üyelerine ulaşabilir
- **ClassU** ve **ClassV** sınıflarının nesneleri de **Package1**'deki sınıfların nesnelerinin sadece **public** üyelerine ulaşabilir
- Ayrıca, **ClassV** miras aldığı için **ClassY**'nin **protected** üyelerini devralır



Örnek - II



- `ClassU` ve `ClassV` sınıfları, aynı pakette olduklarından, birbirlerinin **private** olmayan **static** üyelerine ulaşabilir
- `ClassU` ve `ClassV` sınıflarının nesneleri, aynı pakette olduklarından, birbirlerinin **private** olmayan üyelerine ulaşabilirler
- Hiçbir sınıf ya da nesne, başka hiçbir sınıfın **private** olan **static** ya da nesne üyesine ulaşamaz



API, Application Programming Interface



- API, bir sınıfın/nesnenin (ya da sistemin) dışarıdan ulaşılabilen özelliklerine denir
- API, bir sınıfın (ya da sistemin) `public` olan üyelerinden oluşur
 - Miras amacıyla `protected` olan üyeler de API'ye dahildir
- Dolayısıyla bir sınıfın API'si, `public` ve `protected` üyeleridir
- Bir yazılım sisteminin API'si, `public` olan sınıflarının `public` ve `protected` üyeleridir

public Olmayan Sınıf



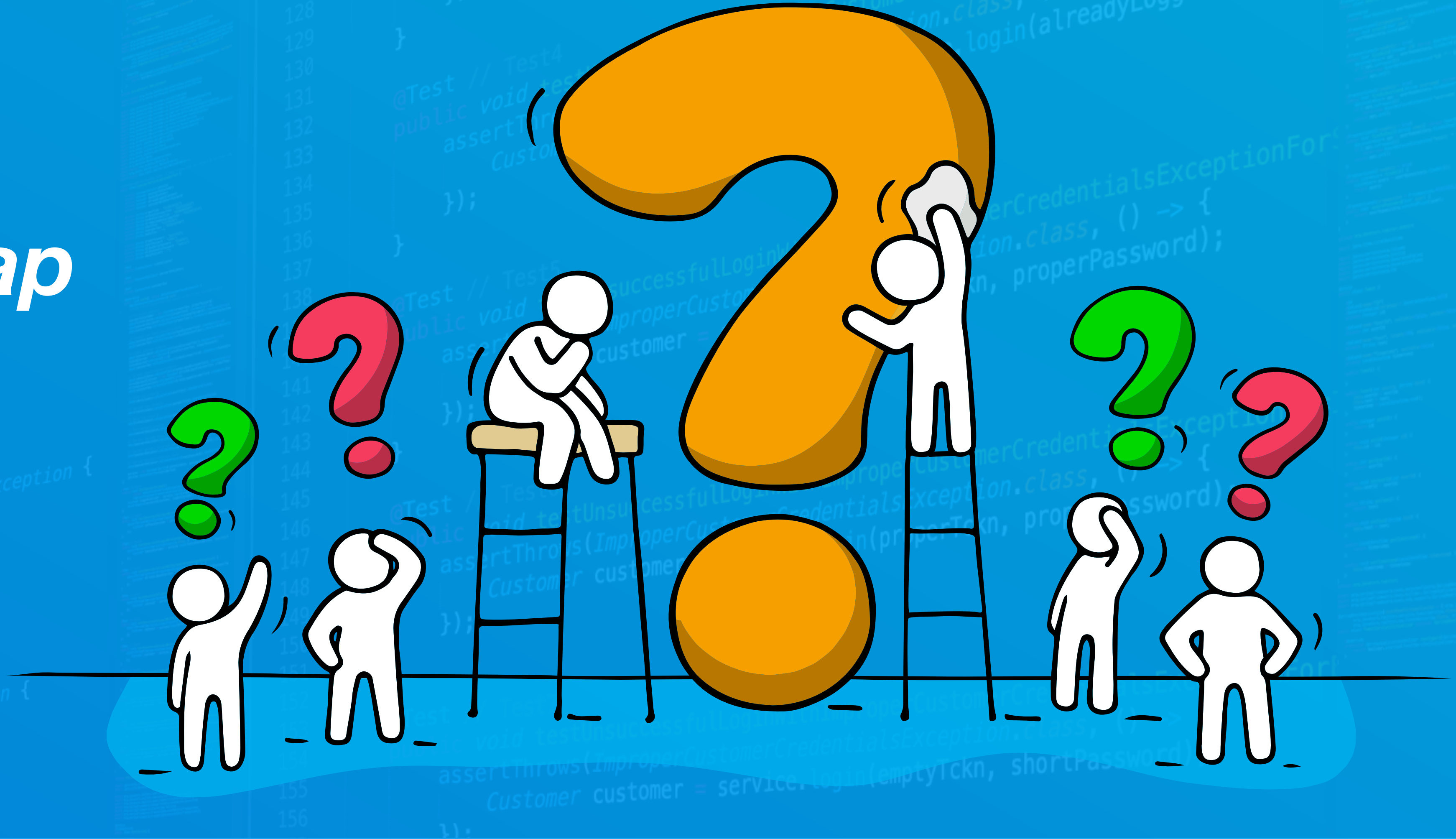
- Eğer bir sınıf `public` değilse (**non-public**) zaten paketi dışından görülemeyeceği için, üyelerinin `public` olmasına gerek yoktur
- Böyle bir sınıfın üyeleri varsayılan ya da `private` erişime sahip olabilir

Üyeleri `private` Sınıf



- Tüm üyeleri `private` erişime sahip olan bir sınıf anlamsızdır
 - Böyle bir sınıf, hiçbir parçası dışarıdan erişilemeyeceğinden, bir sistemin parçası olamaz
- Dolayısıyla makul olan bir sınıfın az ya da çok bazı üyelerinin, örneğin metotlarının varsayılan erişime sahip ya da `public` olmasıdır

Soru ve Cevap Zamanı!



get/set Metotlari



Durumu Dışarıdan Saklı Sınıf - I



- Tüm üye değişkenleri `private` olan bir sınıf düşünün
- Böyle bir nesnenin durumu dışarıdan tamamen saklanmıştır, erişilemez
- Bu sınıfın ve nesnelerinin değişkenlerinin değerlerine ulaşma ihtiyacı olduğunda ne yapılmalıdır?

Durumu Dışarıdan Saklı Sınıf - II



- Örneğin bu nesnenin durumu bir web/mobil vb. arayüzünde gösterilmek istendiğinde ya da buralardan girilen değerlerin nesneye aktarılması gerektiğinde ne olacak?
- Ya da nesnenin durumunun bir yerde, çoğunlukla bir veri tabanında, saklanması gerektiğinde ya da veri tabanından getirilecek değerlerle yeni bir nesnenin yaratılması istendiğinde?
- Benzer şeyleri bir web servis/web API ile yapmak gerektiğinde?

Durumu Dışarıdan Saklı Sınıf - III



- Böyle durumlarda nesnenin durumunu dışarıya açma ihtiyacı kaçınılmazdır
- Önemli olan bunun nasıl yapılacağıdır
- Böyle hallerde değişkenler `public` mi yapılmalıdır?
- Hayır, değişkenler `public` yapılmazlar, `private` olarak bırakılır ama onların değerlerini dışarıya ileten `get` metotları ile dışarıdan değerleri alıp nesneye ileten `set` metotları kullanılır

Set ve Get Metotları - I



- **get/set metotları (getters & setters)** nesnenin durumuna ulaşmak ya da onu değiştirmek amacıyla kullanılan standart metotlardır
- Örneğin eğer nesne değişkeni `int field` ise, **get** ve **set** metotlarının arayüzleri şöyle olur:
 - `int getField()`
 - `void setField(int field)`
- **boolean** alanların **get** metodu `isField` da olabilir

Set ve Get Metotları - II



```
public class Window {
    private boolean open;
    private int width;
    private int height;

    public Window(boolean open, int width, int height){
        this.open = open;
        this.width = width;
        this.height = height;
    }

    public boolean isOpen() {
        return open;
    }

    public void setOpen(boolean open) {
        this.open = open;
    }

    public int getWidth() {
        return width;
    }

    public void setWidth(int width) {
        this.width = width;
    }

    ..
}
```

Window



- `setget.Window`

private Alan ve set/get Metotları



- Alanlar `public` yapılarak doğrudan erişime açılmamalı, `private` olarak bırakılmalı ama gerekiyorsa `set/get` metotlarıyla erişilebilir hale getirilmelidir
- Çünkü doğrudan erişim halinde sınıf ile istemcileri arasındaki bağılılık (coupling) artar
- Ama `set/get` metotları kullanılırsa bağılılık nispeten daha düşük düzeyde tutulabilir

Ne Zaman set/get Metotları - I



- Örneğin aşağıdaki durumlar **set/get** metotlarının rolünü ve onlara olan ihtiyacı ortaya koyar:
- alanların isim, tip, erişim vb. özelliklerinde yapılacak değişiklikler istemcilerde de değişiklik gerektirecektir, **set/get** metotları bu değişikliklerin istemcilere olan etkisini azaltır
- Böyle durumlarda **set/get** metotları, değişikliğin etkisini ortadan kaldıracak ya da en az seviyede tutacak şekilde dönüşümler yapabilir

Ne Zaman set/get Metotları - II



- `set/get` metotlarıyla
- erişim kısıtlamaları yapılabilir
 - Belli özellikleri sağlayan istemcilere, örneğin erişim güvenliğini sağlayanlara veri iletilir
- erişim istekleri sıraya konabilir, böylece örneğin aynı anda birden fazla istemcinin aynı nesne üzerindeki alanı değiştirmesinden ya da ulaşmasından kaynaklanabilecek tutarsızlıklar önlenir

Ne Zaman set/get Metotları - II



- `set` metotlarında, alana geçilen değerlerin doğruluk, tutarlılık, geçerlilik, güvenlik gibi kontrolleri yapılabilir
- `set/get` metotlarında, loglama vb. işler yapılabilir
- gerçekte nesne üzerinde böyle bir alan bile olmayabilir, değer hesaplanıyor ya da başka bir yerde tutuluyor olabilir ama hâlâ `set/get` metotları gereklidir

Ne Zaman set/get Metotları - IV



- `set` metotları, nesnenin varsayılan kurucuyla yaratılıp sonrasında durumunun zamanla inşa edildiği (build) hallerde de kaçınılmaz olabilir
- Sıklıkla, nesnenin durumunun tamamı, nesne yaratılırken bilinemez; bu durumda nesnenin durumunun geri kalanı daha sonra oluşturulur

set/get Metotlarının Problemleri - I



- `set/get` metotlarının negatif tarafı, nesnenin durumunu gereksiz bir şekilde ulaşılabilir kılmasından dolayı artan bağılıktan kaynaklanan karmaşıklıktır
- `set/get` metotlarının gereksiz veya aşırı kullanımı, nesneler arasında bilgi alışverişini artırır
- Çünkü `set/get` metotları iş ya da hizmet değil bilgi alışverişi sağlayan yapılardır

set/get Metotlarının Problemleri - II



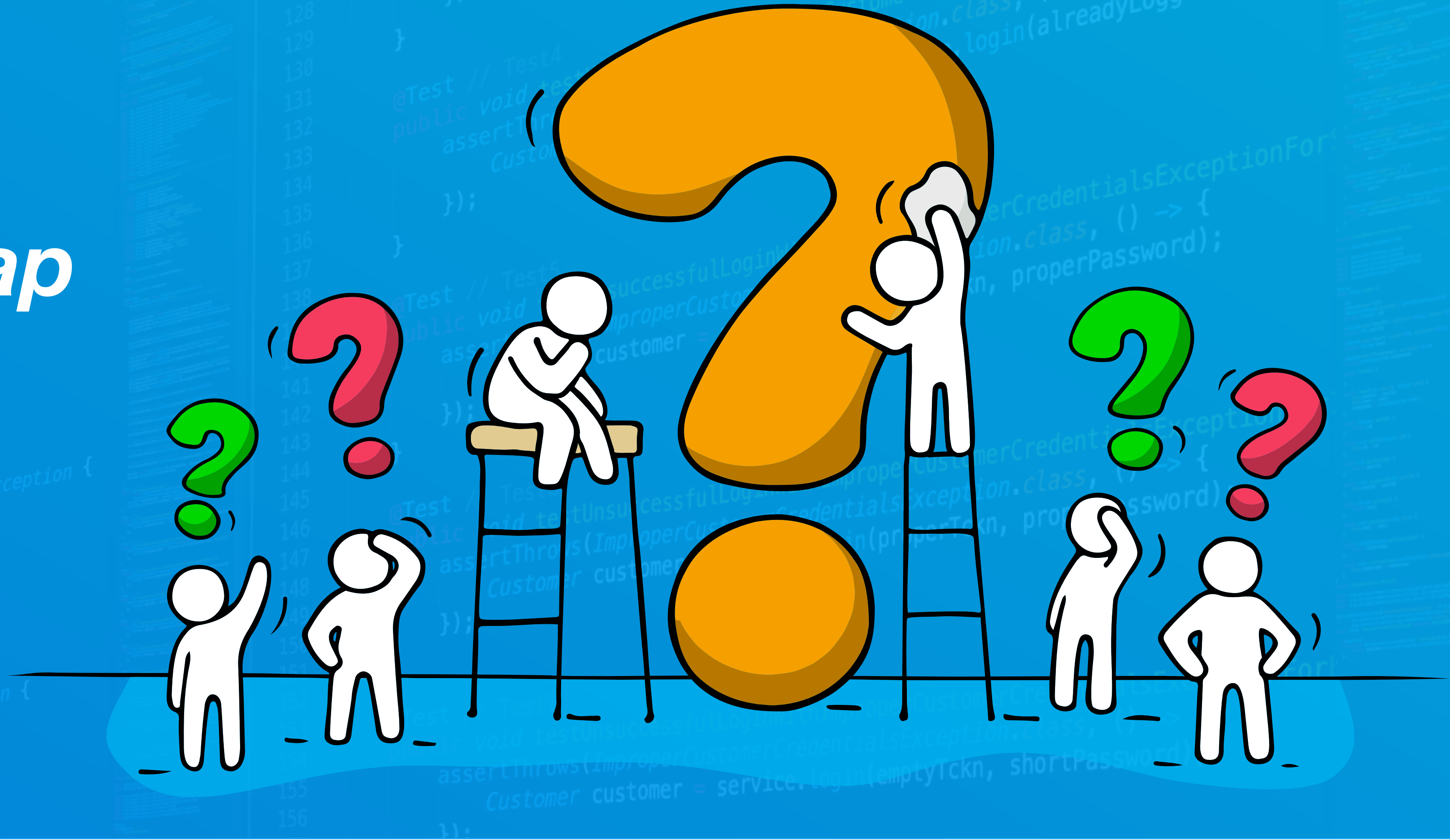
- Arayüz-gerçekleştirme ayrımı ve *program to an interface, not an implementation* gibi prensipler, nesneler arasındaki iletişimin sadece hizmet iletişimi olması gerektiğini ifade eder
- **get** metotları bu konuda daha masum ve gerekli görülebilir ama **set** metotları daha problemlidir çünkü nesnenin durumunu dışarıya, değiştirilebilir olarak sunarlar
- Bu sebeple bahsettiğimiz cinsten ihtiyaçlar dışında **set/get** metotlarından kaçınmak gereklidir

set/get Metotlarının Problemleri - III



- `set/get` metotları tipik olarak, nesneden `get` ile bilgi alıp işleyen ve sonra da sonucunu `set` ile nesneye geri gönderen programlama yaklaşımının en temel ögesidir
- Bu durumda `set/get` metotlarına sahip sınıf hizmet yerine veri sunar ve istemcileri ile arasındaki bağılılık yükselir
- Programlamada böyle bir yaklaşım problemlidir

Soru ve Cevap Zamanı!



Ödevler





1. `setget.Window` sınıfını göz önüne alarak aşağıdaki durumların hangisi nesne merkezli programlama açısından daha sağlıklıdır?

Tartışın:

- a. Aynı işi yapan iki metot, `WindowTest`'in `printWindowInfo()` ve `Window`'un `printInfo()` metotları
- b. Aynı işi yapan iki metot, `WindowTest`'in `calculateWindowArea()` ve `Window`'un `calculateArea()` metotları



2. `setget.Window` sınıfını göz önüne alarak aşağıdaki metotların hangisi nesne merkezli programlama açısından daha sağlıklıdır?

Tartışın:

- a. Aynı işi yapan `Window`'un `setOpen()` metodu ile `open()` ve `close()` metotları



3. `double` tipinde bir `temperature` alanına sahip bir `Thermostat` sınıfı oluşturun. Bu sınıfın nesnelerinin `temperature` alanına değer atanmasına izin vermelidir fakat bu atanan değer sadece -5 ile 40 derece arasında olmasını sağlayın, farklı değerler için nesne hata vermelidir
 - a. Hatayı anormal bir değer döndürerek ve/veya konsola basarak gösterebilirsiniz.



4. `private java.util.Date appointmentDate` alanına ve ilgili kurucuya sahip `Schedule` isimli bir sınıf oluşturun. Bu sınıfa koyacağınız bir `getAppointmentDate()` metoduyla bu alanı döndürmesini sağlayın. Sonra `ScheduleTest` sınıfının `main` metodunda şu satırları yazın:

```
Date myDate = schedule.getAppointmentDate();
```

```
myDate.setTime(0); // Do you think this is ok?
```

- a. Sizce bu son satır sağlıklı mı? Bunun önüne geçmek için `getAppointmentDate()` metodunda ne yapmak gerekir?



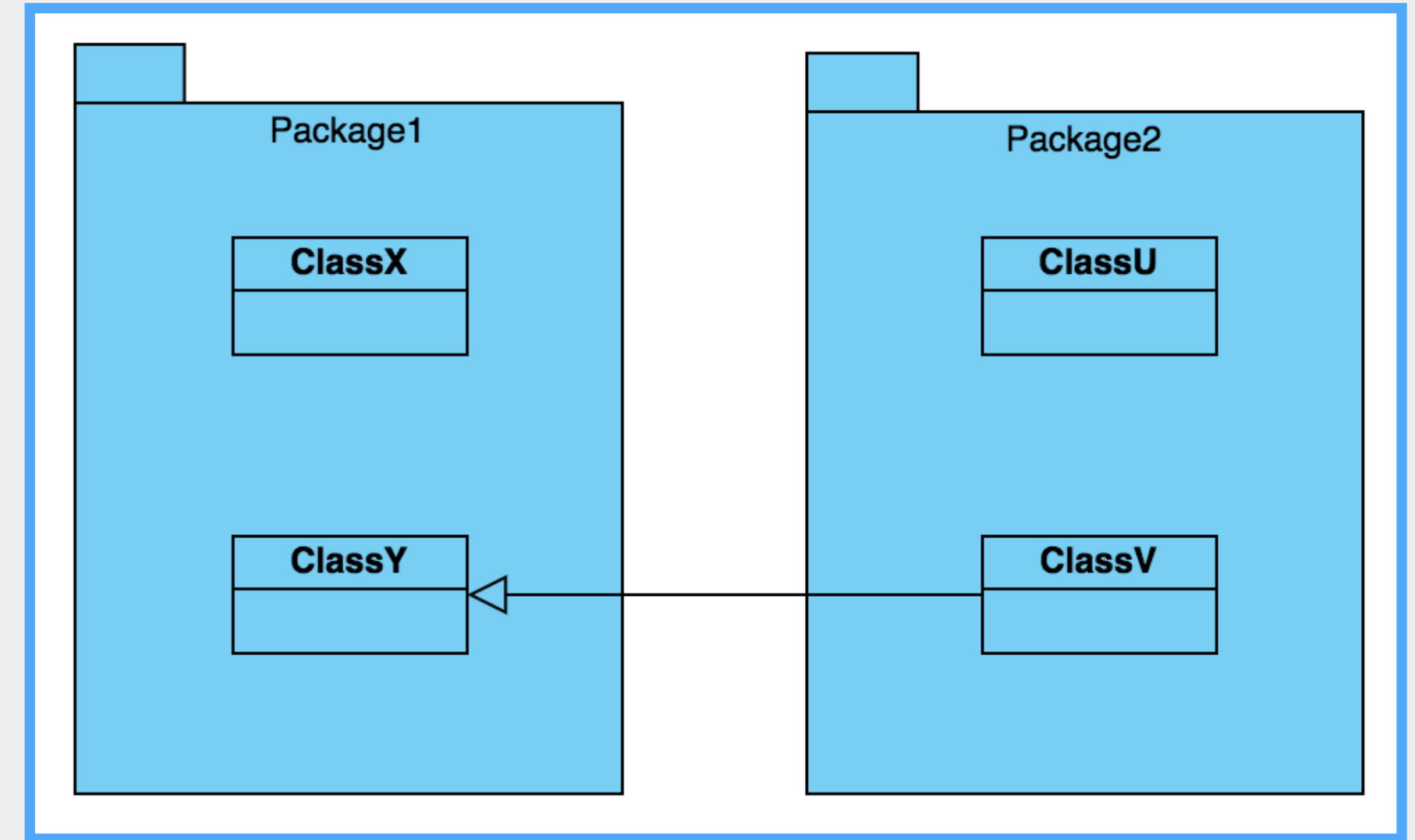
b. `appointmentDate` alanının tipini `java.time.LocalDate` yapın ve farkı gözlemleyin.



5. `String accountNumber`, `double balance` ve `String pin` alanlarına sahip bir `BankAccount` sınıfını gerekli kurucusuyla oluşturun. `accountNumber` değiştirilemez (immutable, read-only), `pin` okunamaz (write-only) olmalıdır. `balance` ise sadece `deposit()` ve `withdraw()` metotlarıyla değiştirilebilmelidir. Ayrıca bu sınıfın üzerine koyacağınız `printInfo()` metoduyla sınıfın nesnesinin durumu hakkında bilgi vermesini sağlayın



6. Yandaki şekilde örnekte verilen paket ve sınıfları oluşturup, **Test** sınıfının **main** metodunda birbirlerine olan erişimlerini test edin





7. Bir sınıf düşünün ve mümkünse üzerinde `public` olabilecek değişkenler ile `private` olabilecek metotlara örnekler verin
 - a. Benzer şekilde varsayılan erişime sahip üyelere örnekler verin

Bundan Sonra?



Bundan Sonra?



- Bundan sonra enum tipini ele alacağız

Bölüm Sonu

Soru ve Cevap Zamanı!

